VERSUS Project Final Report

Khoa Cao

Due 6/23/2026

The Github repository containing the complete code and instructions on how to run the application is at https://www.github.com/koacow/versus.

1 What I Implemented

I extended the Versus skeleton code by adding the tables for predictions, votes, achievements, follows, and comments. I also created the two BEFORE INSERT triggers to enforce the integrity constraints for votes and predictions. Additionally, I added the AFTER INSERT trigger that awards achievements to users upon creating their first bracket and submitting their 10th prediction. I also implemented the stored procedure that closes a round, scores the predictions, advances the winner, and updates the bracket status. This procedure is implemented as a single transaction to ensure data consistency between the Bracket and Matchups tables in the event of any errors.

On the application side, I implemented the necessary routes and logic for user interaction as per the specifications, created the necessary HTML templates, and created a simple and clean user interface using CSS (I have had extensive experience with CSS in previous projects). An extra feature I added was the ability for users to search for other users, view their profiles, and follow them.

2 A Transaction

The add-vote transaction involves inserting a new vote into the Votes table while simultaneously updating the vote count for the selected entrant in the Matchups table. This must be atomic to ensure that a vote the number of votes for an entrant in a matchup in the Votes table is consistent with the recorded number of votes for an entrant in the Matchups table. The transaction logic, which uses MySQL's transactional capabilities, is shown below.

```
@app.route('/bracket<int:bracket_id>/vote/<int:matchup_id>', methods=['POST'])
@flask_login.login_required
def vote(bracket_id, matchup_id):
    uid       = getUserIdFromUsername(flask_login.current_user.id)
    entrant_id = request.form.get('entrant_id')
    cursor = conn.cursor()
    try:
        # Determine which counter to bump before acquiring any lock
        cursor.execute(
            "SELECT entrant_a_id, entrant_b_id FROM Matchups WHERE matchup_id = '{0}'".form
        row = cursor.fetchone()
        if not row:
            raise ValueError("Matchup not found")
        a_id, b_id = row
```

```
            cursor.execute("START TRANSACTION;")
            cursor.execute("""
            INSERT INTO Votes (user_id, matchup_id, voted_for_entrant_id) VALUES ('{0}', '{1}'
            """.format(uid, matchup_id, entrant_id))
            if int(entrant_id) == a_id:
                cursor.execute("UPDATE Matchups SET votes_a = votes_a + 1 WHERE matchup_id = '
            else:
                cursor.execute("UPDATE Matchups SET votes_b = votes_b + 1 WHERE matchup_id = '
            cursor.execute("COMMIT;")
            conn.commit()
        except mysql.connector.Error:
            conn.rollback()
        cursor.close()
        return redirect(url_for('view_bracket', bracket_id=bracket_id))
```

3 Triggers

Prediction gate

```
CREATE TRIGGER prediction_open_check
BEFORE INSERT ON Predictions
FOR EACH ROW
BEGIN
    IF (
        SELECT status
        FROM Brackets
        WHERE bracket_id = (SELECT bracket_id FROM Matchups WHERE matchup_id = NEW.matchup_id)
        ) NOT IN ('predictions_open') THEN
        SIGNAL SQLSTATE '45000' SET MESSAGE_TEXT = 'Predictions are closed for this bracket';
    END IF;
END
```

This trigger blocks any prediction inserted after the bracket has left the predictions_open state, and it lives in the database so the rule is enforced consistently even if the Flask app is bypassed.

Vote gate

```
CREATE TRIGGER vote_current_round_check
BEFORE INSERT ON Votes
FOR EACH ROW
BEGIN
    DECLARE current_round INT;
    SET current_round = (SELECT round FROM Matchups WHERE matchup_id = NEW.matchup_id);
    IF (SELECT status FROM Brackets WHERE bracket_id = (SELECT bracket_id FROM Matchups WHERE m
        SIGNAL SQLSTATE '45000' SET MESSAGE_TEXT = 'Voting is not open for this matchup';
```

```
    END IF;
END
```

This trigger blocks votes for a matchup unless the bracket is currently in the matching round state. Similar to the prediction gate, it belongs in the database because it protects data integrity at the storage layer, ensuring that the voting rules are consistently enforced across all access points to the database.

4 Database Schema

Table 1: Users

Column	Type	Key	Constraints
user_id	INT	PK	AUTO_INCREMENT
username	VARCHAR(50)		NOT NULL, UNIQUE
email	VARCHAR(255)		NOT NULL, UNIQUE
password	VARCHAR(255)		NOT NULL
bio	TEXT		NULL
created_at	DATETIME		NOT NULL, DEFAULT CURRENT_TIMESTAMP

Table 2: Brackets

Column	Type	Key	Constraints
bracket_id	INT	PK	AUTO_INCREMENT
host_id	INT	FK	NOT NULL, FK → Users(user_id)
title	VARCHAR(255)		NOT NULL
description	TEXT		NULL
entrant_count	INT		NOT NULL, CHECK IN (4, 8, 16, 32)
status	ENUM('draft', 'predictions_open', 'round_1', 'round_2', 'round_3', 'round_4', 'round_5', 'completed')		NOT NULL, DEFAULT 'predictions_open'
created_at	DATETIME		NOT NULL, DEFAULT CURRENT_TIMESTAMP
prediction_deadline	DATETIME		NULL

Table 3: Entrants

Column	Type	Key	Constraints
entrant_id	INT	PK	AUTO_INCREMENT
bracket_id	INT	FK	NOT NULL, FK → Brackets(bracket_id), UNIQUE (bracket_id, seed)
seed	INT		NOT NULL, UNIQUE (bracket_id, seed)
name	VARCHAR(255)		NOT NULL
img_url	VARCHAR(255)		NULL

Table 4: Matchups

Column	Type	Key	Constraints
matchup_id	INT	PK	AUTO_INCREMENT
bracket_id	INT	FK	NOT NULL, FK → Brackets(bracket_id), UNIQUE (bracket_id, round, slot)
round	INT		NOT NULL, UNIQUE (bracket_id, round, slot)
slot	INT		NOT NULL, UNIQUE (bracket_id, round, slot)
entrant_a_id	INT	FK	FK → Entrants(entrant_id)
entrant_b_id	INT	FK	FK → Entrants(entrant_id)
winner_entrant_id	INT	FK	FK → Entrants(entrant_id)
votes_a	INT		NOT NULL, DEFAULT 0
votes_b	INT		NOT NULL, DEFAULT 0

Table 5: Predictions

Column	Type	Key	Constraints
prediction_id	INT	PK	AUTO_INCREMENT
user_id	INT	FK	NOT NULL, FK → Users(user_id)
matchup_id	INT	FK	NOT NULL, FK → Matchups(matchup_id)
predicted_winner_id	INT	FK	NOT NULL, FK → Entrants(entrant_id)
submitted_at	DATETIME		NOT NULL, DEFAULT CURRENT_TIMESTAMP
is_correct	BOOLEAN		NULL
points_earned	INT		NULL

Table 6: Votes

Column	Type	Key	Constraints
vote_id	INT	PK	AUTO_INCREMENT
user_id	INT	FK	NOT NULL, FK → Users(user_id)
matchup_id	INT	FK	NOT NULL, FK → Matchups(matchup_id)
voted_for_entrant_id	INT	FK	NOT NULL, FK → Entrants(entrant_id), UNIQUE (user_id, matchup_id)

Table 7: Achievements

Column	Type	Key	Constraints
achievement_code	ENUM('bracket_maker', 'locked_in')	PK	PRIMARY KEY
name	VARCHAR(255)		NOT NULL
description	TEXT		NOT NULL

Table 8: User_Achievements

Column	Type	Key	Constraints
user_id	INT	PK/FK	NOT NULL, FK → Users(user_id), part of composite PK
achievement_code	ENUM('bracket_maker', 'locked_in')	PK/FK	NOT NULL, FK → Achievements(achievement_code), part of composite PK
achieved_at	DATETIME		NOT NULL, DEFAULT CURRENT_TIMESTAMP

Table 9: Follows

Column	Type	Key	Constraints
follower_id	INT	PK/FK	NOT NULL, FK → Users(user_id), part of composite PK
followed_id	INT	PK/FK	NOT NULL, FK → Users(user_id), part of composite PK, CHECK follower_id $\neq$ followed_id
followed_at	DATETIME		NOT NULL, DEFAULT CURRENT_TIMESTAMP

Table 10: Comments

Column	Type	Key	Constraints
comment_id	INT	PK	AUTO_INCREMENT
user_id	INT	FK	NOT NULL, FK → Users(user_id)
matchup_id	INT	FK	NOT NULL, FK → Matchups(matchup_id)
body	TEXT		NOT NULL
created_at	DATETIME		NOT NULL, DEFAULT CURRENT_TIMESTAMP